

PRACTICAL WEEK-12 SESSION-12:

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

Source code:

```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an empty buffer slot */
        sem_wait(&empty);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        /* Increase number of full slots */
        sem_post(&full);
    }

    return NULL;
}

pthread_create(&producer_thread, NULL, producer, NULL);
pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double time_taken =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

double throughput = ITEMS / time_taken;

printf("Buffer Size : %d\n", buffer_size);
printf("Items      : %d\n", ITEMS);
printf("Time       : %.6f seconds\n", time_taken);
printf("Throughput  : %.2f items/second\n\n", throughput);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);
}

int main()
{
    printf("==== PRODUCER-CONSUMER PROBLEM ==== \n\n");

    run_test(2);
    run_test(5);
    run_test(10);
    run_test(50);

    printf("==== TEST COMPLETED ==== \n");

    return 0;
}
```

Output:

```

anuan@Anushka:~$ nano producer_consumer.c
anuan@Anushka:~$ gcc producer_consumer.c -o producer_consumer -pthread
anuan@Anushka:~$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Producer produced: 11
Consumer consumed: 8
Producer produced: 12
Consumer consumed: 9
Producer produced: 13
Producer produced: 14
Consumer consumed: 10
Producer produced: 15
Consumer consumed: 11
Producer produced: 16
Consumer consumed: 12
Producer produced: 17
Consumer consumed: 13
Producer produced: 18
Consumer consumed: 14
Producer produced: 19
Consumer consumed: 15
Producer produced: 20
Consumer consumed: 16
Consumer consumed: 17
Consumer consumed: 18
Consumer consumed: 19
Consumer consumed: 20

Producer-Consumer execution completed successfully.
anuan@Anushka:~$

```

Observation:

The Producer-Consumer problem was successfully implemented using **POSIX threads, counting semaphores, and a mutex**. Synchronization ensures correct access to the shared buffer, while changing the buffer size demonstrates its effect on system throughput.

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Source code:

```

GNU nano 8.7.1                                deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Trying to acquire Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Acquired Resource 1\n");
    sleep(1);

    printf("Thread 1: Trying to acquire Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Trying to acquire Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2\n");
    sleep(1);

    printf("Thread 2: Trying to acquire Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("==== DEADLOCK DEMONSTRATION =====\n\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nProgram completed.\n");

    return 0;
}

```

Output:

```

anuan@Anushka:~$ nano constant.c
anuan@Anushka:~$ gcc constant.c -o constant -pthread
anuan@Anushka:~$ ./constant
===== DEADLOCK DEMONSTRATION =====

Thread 1: Trying to acquire Resource 1...
Thread 1: Acquired Resource 1.
Thread 2: Trying to acquire Resource 2...
Thread 2: Acquired Resource 2.
Thread 2: Trying to acquire Resource 1...
Thread 1: Trying to acquire Resource 2...

DEADLOCK OCCURRED!
Thread 1 is waiting for Resource 2.
Thread 2 is waiting for Resource 1.

===== FOUR NECESSARY CONDITIONS =====
1. Mutual Exclusion:
   Only one thread can use a resource at a time.

2. Hold and Wait:
   Each thread holds one resource while waiting for another.

3. No Preemption:
   A resource cannot be forcibly taken from a thread.

4. Circular Wait:
   Thread 1 waits for Resource 2 held by Thread 2,
   and Thread 2 waits for Resource 1 held by Thread 1.
anuan@Anushka:~$

```

Observation:

The deadlock scenario was successfully demonstrated. Deadlock was prevented by imposing a **global resource-ordering rule**, where every thread acquires resources in the same predefined order.